Here are some practice questions along with sample database schemas and data for each of the SQL topics you mentioned. These questions are designed to help you get hands-on practice with writing SQL queries.

---

1. SELECT * FROM

Table: `employees`

emp_id	emp_name	department	salary
1	John Doe	Sales	60000
2	Jane Smith	HR	75000
3	Mark Taylor	IT	80000
4	Emma Wilson	Sales	65000

Question:

- Write a query to fetch all columns for all employees.

Sample Query:

SELECT * FROM employees;

---

2. SELECT DISTINCT

Table: `orders`

order_id	customer_name	product	quantity
1	Alice	Laptop	1
2	Bob	Smartphone	2
3	Alice	Laptop	1

4	Charlie	Tablet	1

Question:

- Fetch the unique product names from the orders table.

Sample Query:

```
SELECT DISTINCT product FROM orders;
```

---

3. COUNT

Table: `users`

user_id	user_name	status
1	Alex Johnson	Active
2	Brittany Brown	Inactive
3	Chris Evans	Active
4	Daniel Lee	Active

Question:

- Count the number of active users.

Sample Query:

```
SELECT COUNT(*) AS active_users FROM users WHERE status = 'Active';
```

---

4. Alias (AS)

Question:

- Fetch the employee names and their salaries with column aliases "Name" and "Income".

Sample Query:

SELECT emp_name AS Name, salary AS Income FROM employees;

---

5. WHERE

Table: `products`

product_id	product_name	price
1	Keyboard	25
2	Mouse	15
3	Monitor	150
4	Laptop	800

Question:

- Retrieve products priced above 50.

Sample Query:

SELECT * FROM products WHERE price > 50;

---

6. LIMIT and ORDER BY

Question:

- Fetch the top 2 highest paid employees.

Sample Query:

SELECT emp_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 2;

---

7. Operators and Filtering

Question:

- Get employees who are either in Sales or have a salary above 70,000.

Sample Query:

```
SELECT *
FROM employees
WHERE department = 'Sales' OR salary > 70000;
```

---

8. BETWEEN, IN, LIKE, and ILIKE

Questions:

- Fetch products with a price between 20 and 100.
- Retrieve orders where the product is either 'Laptop' or 'Tablet'.
- Find employee names starting with 'J'.
- Case insensitive search for employee names containing 'son'.

Sample Queries:

```
SELECT * FROM products WHERE price BETWEEN 20 AND 100;

SELECT * FROM orders WHERE product IN ('Laptop', 'Tablet');

SELECT * FROM employees WHERE emp_name LIKE 'J%';

SELECT * FROM employees WHERE emp_name ILIKE '%son%';
```

---

9. CASE, IF, ELSE

Question:

- Display employee names along with their salary category as 'High' if above 70,000, else 'Low'.

Sample Query:

```
SELECT emp_name,
   CASE
      WHEN salary > 70000 THEN 'High'
      ELSE 'Low'
   END AS salary_category
FROM employees;
```

---

10. COALESCE and NULLIF

Table: `orders_v2`

order_id	customer_name	delivery_date
1	Alice	2025-02-20
2	Bob	NULL
3	Charlie	2025-02-22

Questions:

- Show the delivery date, but if it is NULL, display 'Pending'.
- Compare two columns and return NULL if they are the same.

Sample Queries:

```
SELECT order_id,
   COALESCE(delivery_date, 'Pending') AS delivery_status
FROM orders_v2;

SELECT product_name,
   NULLIF(price, 0) AS price
FROM products;
```

Here are more practice questions for each topic, designed to deepen your understanding of SQL queries. They continue to use the tables from the previous examples and introduce a few new ones.

---

1. SELECT * FROM

Question:

- Fetch all details of employees who belong to the 'Sales' department.

Sample Query:

```
SELECT * FROM employees WHERE department = 'Sales';
```

---

2. SELECT DISTINCT

Table: `transactions`

trans_id	customer_id	payment_mode	amount
1	101	Credit Card	500
2	102	Debit Card	200
3	101	UPI	150
4	103	Credit Card	800
5	104	Cash	300

Questions:

- List all unique payment modes used in transactions.

Sample Query:

```
SELECT DISTINCT payment_mode FROM transactions;
```

---

3. COUNT

Question:

- Count the number of employees in each department.

Sample Query:

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

---

4. Alias (AS)

Question:

- Display employee names and their salary incremented by 10% as "New Salary".

Sample Query:

```
SELECT emp_name,
    salary * 1.10 AS "New Salary"
FROM employees;
```

---

5. WHERE

Table: `students`

student_id	student_name	grade	score
1	Lisa	A	85
2	Sam	B	70
3	Mark	A	92
4	Tom	C	65

Questions:

- Get details of students who scored above 80 and are in grade 'A'.

Sample Query:

```
SELECT * FROM students
WHERE score > 80 AND grade = 'A';
```

---

6. LIMIT and ORDER BY

Questions:

- Retrieve the names of the top 3 highest scoring students.

Sample Query:

```
SELECT student_name, score
FROM students
ORDER BY score DESC
LIMIT 3;
```

---

7. Operators and Filtering

Question:

- Get the employees who are either in 'HR' or 'IT' department and earn more than 70,000.

Sample Query:

```
SELECT * FROM employees
WHERE department IN ('HR', 'IT')
AND salary > 70000;
```

---

8. BETWEEN, IN, LIKE, and ILIKE

Questions:

- Fetch products with prices between 100 and 500.
- Retrieve employees whose names end with 'son'.
- Find orders where product is either 'Tablet' or 'Smartphone'.
- Case insensitive search for customer names containing 'li'.

Sample Queries:

```
SELECT * FROM products WHERE price BETWEEN 100 AND 500;

SELECT * FROM employees WHERE emp_name LIKE '%son';

SELECT * FROM orders WHERE product IN ('Tablet', 'Smartphone');

SELECT * FROM orders WHERE customer_name ILIKE '%li%';
```

---

9. CASE, IF, ELSE

Question:

- Show the product name and its availability status as 'In Stock' if the quantity is more than 0, else 'Out of Stock'.

Table: `inventory`

product_id	product_name	quantity
1	Laptop	5
2	Keyboard	0
3	Mouse	10

Sample Query:

```
SELECT product_name,
   CASE
      WHEN quantity > 0 THEN 'In Stock'
      ELSE 'Out of Stock'
   END AS availability
FROM inventory;
```

---

10. COALESCE and NULLIF

Questions:

- Display customer names and delivery dates, but if the delivery date is NULL, show 'Not Delivered'.
- Compare the score of students with 0 and return NULL if the score is 0.

Sample Queries:

```
SELECT customer_name,
    COALESCE(delivery_date, 'Not Delivered') AS delivery_status
FROM orders_v2;

SELECT student_name,
    NULLIF(score, 0) AS score
FROM students;
```

---

11. Advanced Challenges

1. Combining Concepts:

- Retrieve all products whose names contain the letter 'a' (case insensitive) and are priced between 50 and 200, ordered by price in ascending order.

```
SELECT * FROM products
WHERE product_name ILIKE '%a%'
AND price BETWEEN 50 AND 200
ORDER BY price ASC;
```

2. Using CASE with Aggregate:

- Count the number of students in each grade and label it as 'High Count' if more than 1 student, otherwise 'Low Count'.

```
SELECT grade,
    COUNT(*) AS student_count,
    CASE
      WHEN COUNT(*) > 1 THEN 'High Count'
      ELSE 'Low Count'
    END AS count_category
FROM students
GROUP BY grade;
```

3. Using COALESCE and NULLIF Together:

- Display employee names and their bonus (salary * 0.1). If the salary is NULL, show the bonus as 0. If the bonus is 0, show it as NULL.

```
SELECT emp_name,
    NULLIF(COALESCE(salary * 0.1, 0), 0) AS bonus
FROM employees;
```

Here are more advanced practice questions covering each of the SQL topics. These are designed to challenge your understanding and help you master complex query writing.

---

1. SELECT * FROM

Question:

- Fetch all details of employees but exclude those who are in the 'HR' department.

Sample Query:

```
SELECT * FROM employees WHERE department != 'HR';
```

---

2. SELECT DISTINCT

Table: `sales`

sale_id	product_name	region	amount
1	Laptop	North	1200
2	Smartphone	South	800
3	Laptop	East	1100
4	Tablet	North	600
5	Laptop	North	1200

Questions:

- Get a list of unique regions where sales have been made.
- Get distinct product names sold in the 'North' region.

Sample Queries:

```
SELECT DISTINCT region FROM sales;

SELECT DISTINCT product_name
FROM sales
WHERE region = 'North';
```

---

3. COUNT

Questions:

- Count the number of different products sold.
- Count how many employees have salaries above 70,000.

Sample Queries:

```
SELECT COUNT(DISTINCT product_name) AS unique_products_sold
FROM sales;

SELECT COUNT(*) AS high_income_employees
FROM employees
WHERE salary > 70000;
```

---

4. Alias (AS)

Questions:

- Display the product name and its price incremented by 15% as "Discounted Price".
- Fetch employee names and their department with aliases as "Employee" and "Dept".

Sample Queries:

```
SELECT product_name,
    price * 1.15 AS "Discounted Price"
```

```
FROM products;

SELECT emp_name AS "Employee",
       department AS "Dept"
FROM employees;
```

---

5. WHERE

Table: `bookings`

booking_id	customer_name	destination	amount
1	John	Paris	500
2	Emma	London	800
3	Jack	Paris	700
4	Olivia	New York	1200
5	Emma	Paris	650

Questions:

- Fetch bookings for the destination 'Paris' with an amount greater than 600.
- Get employees who are neither in 'IT' nor in 'HR' departments.

Sample Queries:

```
SELECT * FROM bookings
WHERE destination = 'Paris'
AND amount > 600;

SELECT * FROM employees
WHERE department NOT IN ('IT', 'HR');
```

---

6. LIMIT and ORDER BY

Questions:

- Get the 3 lowest paid employees.
- List the top 5 sales by amount in descending order.

Sample Queries:

```
SELECT emp_name, salary
FROM employees
ORDER BY salary ASC
LIMIT 3;

SELECT product_name, amount
FROM sales
ORDER BY amount DESC
LIMIT 5;
```

---

7. Operators and Filtering

Questions:

- Retrieve employees who are in 'Sales' and have a salary between 60000 and 80000.
- Get sales records where the amount is either 800 or 1200.

Sample Queries:

```
SELECT * FROM employees
WHERE department = 'Sales'
AND salary BETWEEN 60000 AND 80000;

SELECT * FROM sales
WHERE amount IN (800, 1200);
```

---

8. BETWEEN, IN, LIKE, and ILIKE

Questions:

- Get products whose names start with 'L' and are priced between 100 and 1000.
- List all customers whose name contains 'm' (case insensitive).

Sample Queries:

```
SELECT * FROM products
WHERE product_name LIKE 'L%'
AND price BETWEEN 100 AND 1000;

SELECT * FROM bookings
WHERE customer_name ILIKE '%m%';
```

---

9. CASE, IF, ELSE

Questions:

- Show employee names and their performance as 'Excellent' if salary > 80000, 'Good' if between 60000 and 80000, otherwise 'Average'.
- Display product names and mark them as 'Expensive' if the price is above 500, otherwise 'Affordable'.

Sample Queries:

```
SELECT emp_name,
   CASE
      WHEN salary > 80000 THEN 'Excellent'
      WHEN salary BETWEEN 60000 AND 80000 THEN 'Good'
      ELSE 'Average'
   END AS performance
FROM employees;

SELECT product_name,
   CASE
      WHEN price > 500 THEN 'Expensive'
      ELSE 'Affordable'
   END AS price_category
FROM products;
```

---

10. COALESCE and NULLIF

Questions:

- Show customer names and booking amounts, but if the amount is NULL, show 'Pending'.

- Compare sales amounts and return NULL if the amount is zero.

Sample Queries:

```
SELECT customer_name,
    COALESCE(amount, 'Pending') AS booking_amount
FROM bookings;

SELECT product_name,
    NULLIF(amount, 0) AS amount
FROM sales;
```

---

11. Advanced Challenges

1. Using Multiple Conditions:

- Fetch all employees who either work in 'IT' and earn above 80000 or are in 'Sales' and earn below 70000.

```
SELECT * FROM employees
WHERE (department = 'IT' AND salary > 80000)
  OR (department = 'Sales' AND salary < 70000);
```

2. Complex CASE:

- Display order details with a column 'Status' as 'High' if amount > 1000, 'Medium' if between 500 and 1000, and 'Low' otherwise.

```
SELECT order_id, customer_name, amount,
  CASE
    WHEN amount > 1000 THEN 'High'
    WHEN amount BETWEEN 500 AND 1000 THEN 'Medium'
    ELSE 'Low'
  END AS status
FROM orders;
```

3. Nested Queries with DISTINCT:

- Find the distinct products sold by customers who have made at least 2 purchases.

```
SELECT DISTINCT product_name
```

```
FROM orders
WHERE customer_name IN (
   SELECT customer_name
   FROM orders
   GROUP BY customer_name
   HAVING COUNT(order_id) >= 2
);
```

4. Combining COALESCE and NULLIF:

- Display employee names and their bonuses (salary * 0.1). If the salary is NULL, the bonus should be 0. If the bonus is 0, show it as NULL.

```
SELECT emp_name,
     NULLIF(COALESCE(salary * 0.1, 0), 0) AS bonus
FROM employees;
```

Would you like even more questions, or do you need help with more advanced topics like JOINS, subqueries, or window functions?